紫紅虎

數位邏輯設計實驗結報

- 實驗班別：■資工(星期三)
- 實驗名稱：(八) 4 位元 ripple 加法器
- 實驗日期： 2025 年 4 月 30 日
- 組別編號： 22
- 成員名單：

【實驗目的】

以 Verilog HDL 硬體描述語言，在 DE2-115 平台上實作 4 位元加法器。

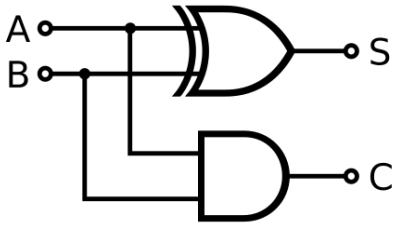
【實驗方法】

實作**半加器** → 使用 2 個半加器實作**全加器** → 使用 4 個全加器實作**四位元加法器**

《半加器》

只能加兩個位元（bit）的數位邏輯電路，輸入為兩個位元 A 和 B，輸出有兩個：

- **總和**（Sum, S）
- **進位**（Carry, C）



Input		Output	
A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

```
module half_adder (output S, C, input x, y);
    xor (S, x, y);
    and (C, x, y);
endmodule
```

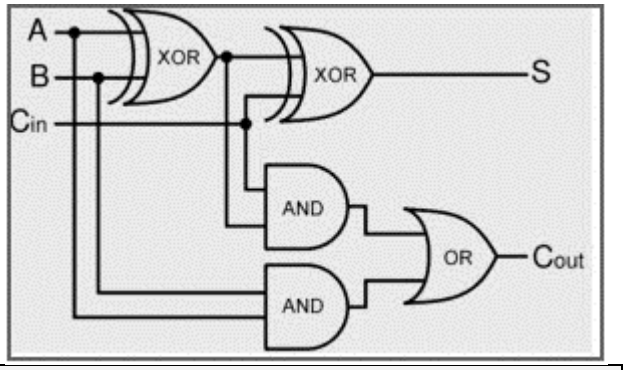
《全加器》

全加器除了加 A 和 B 外，**還能加上來自低位元的進位（Cin）**，共有三個輸入：

- A、B、Cin（進位輸入）

輸出仍是兩個：

- **總和（Sum, S）**
- **進位輸出（Cout）**



Input			Output	
A	B	Cin	Sum	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

```
module full_adder (output S, C, input x, y, z);
    wire S1, C1, C2;
    half_adder HA1 (S1, C1, x, y);
    half_adder HA2 (S, C2, S1, z);
    or G1 (C, C2, C1);
endmodule
```

《四位元加法器》

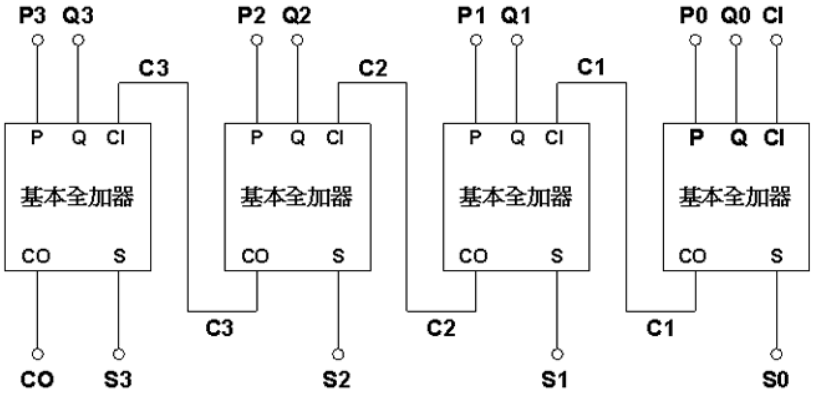
能處理兩個四位元二進位數字相加的電路，會串接四個全加器。

輸入：

- A[3:0]：四位元的第一個加數
- B[3:0]：四位元的第二個加數
- Cin：初始進位（通常設為 0）

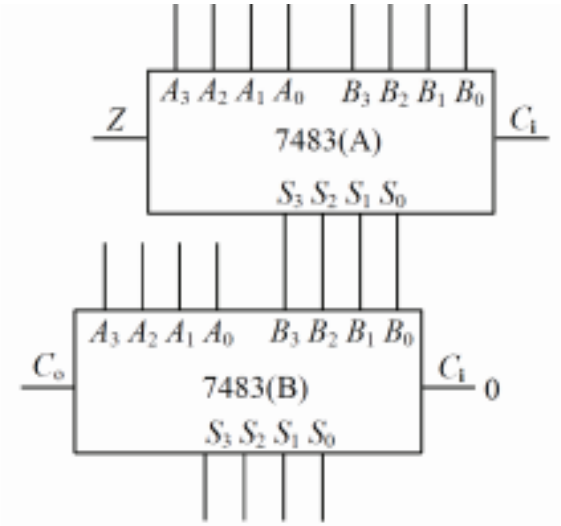
輸出：

- Sum[3:0]：四位元的總和
- Cout：最終進位



《mod_9 加法器》

中間處加入邏輯閘，實現 9_mod→



	k	Z3	Z2	Z1	Z0
0	0	0	0	0	0
1	0	0	0	0	1
2	0	0	0	1	0
3	0	0	0	1	1
4	0	0	1	0	0
5	0	0	1	0	1
6	0	0	1	1	0
7	0	0	1	1	1
8	0	1	0	0	0
9	0	1	0	0	1
10	0	1	0	1	0
11	0	1	0	1	1
12	0	1	1	0	0
13	0	1	1	0	1
14	0	1	1	1	0
15	0	1	1	1	1
16	1	0	0	0	0
17	1	0	0	0	1

mod_9 = 8 + 8 + 1 = 17

Z3 Z2 / Z1 Z0	00	01	11	10
00	0	0	0	0
01	0	0	0	0
11	1	1	1	1
10	0	1	1	1

最佳解：Z3Z2 + Z3Z1 + Z3Z1

（還需與第一個 7483 的 Cout AND）

【程式碼】

```
//半加器

module half_adder (output S, C, input x, y);
    xor (S, x, y);
    and (C, x, y);
endmodule

//全加器

module full_adder (output S, C, input x, y, z);
    wire S1, C1, C2;
    half_adder HA1 (S1, C1, x, y);
    half_adder HA2 (S, C2, S1, z);
    or G1 (C, C2, C1);
endmodule


//mod_9 加法器

module mod9_adder(output [3:0] result, output carry_out, input [3:0] A, B, input Cin);
    wire [3:0] Z;
    wire c1, c2, c3, c4;

//第一個 7483

    full_adder FA0(Z[0], c1, A[0], B[0], Cin);
    full_adder FA1(Z[1], c2, A[1], B[1], c1);
    full_adder FA2(Z[2], c3, A[2], B[2], c2);
    full_adder FA3(Z[3], c4, A[3], B[3], c3);

    //判斷是否超過 9
    wire ge9;
    assign ge9 = c4 | (Z[3] & Z[2]) | (Z[3] & Z[1]) | (Z[3] & Z[0]); //c4 為第一個 7483 的
carry_out，若為 1 則超過 15（>9），其他為 kmap 結果
    assign carry_out = ge9;  //carry_out

//超過 9，用-9（-9 = 0111）
    wire [3:0] nine_comp = 4'b0111; //第二個 7483 的 A4-A0
    wire [3:0] adj_sum; //第二個 7483 的 B4-B0
    wire d1, d2, d3;

//第二個 7483

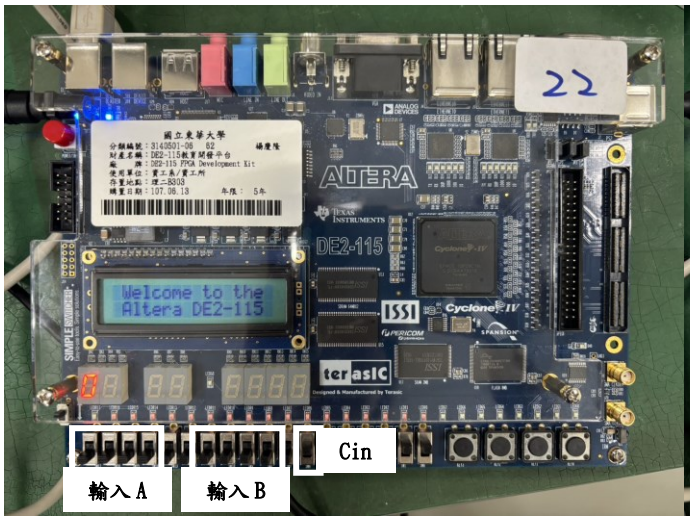
    full_adder SUB0(adj_sum[0], d1, Z[0], nine_comp[0], 1'b0);
    full_adder SUB1(adj_sum[1], d2, Z[1], nine_comp[1], d1);
    full_adder SUB2(adj_sum[2], d3, Z[2], nine_comp[2], d2);
    full_adder SUB3(adj_sum[3],    , Z[3], nine_comp[3], d3);

    assign result = ge9 ? adj_sum : Z;
endmodule
```

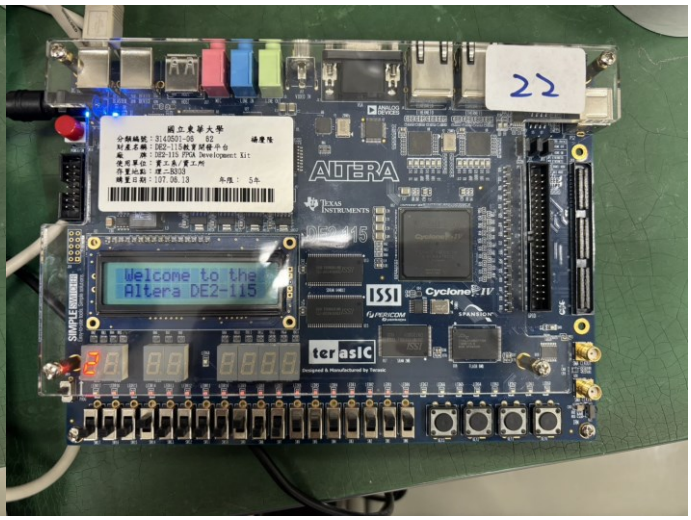
```
module myDecoder(input [3:0] inValue, output reg [6:0] outSeg);
always @(*) begin
    case (inValue)
        4'd0: outSeg = 7'b1000000;
        4'd1: outSeg = 7'b1111001;
        4'd2: outSeg = 7'b0100100;
        4'd3: outSeg = 7'b0110000;
        4'd4: outSeg = 7'b0011001;
        4'd5: outSeg = 7'b0010010;
        4'd6: outSeg = 7'b0000010;
        4'd7: outSeg = 7'b1111000;
        4'd8: outSeg = 7'b0000000;
        default: outSeg = 7'b1111111;
    endcase
end
endmodule
module top(
    input [3:0] sw1,
    input [3:0] sw2,
    input sw_cin,
    output [6:0] seg,
    output ledC
);
    wire [3:0] result;

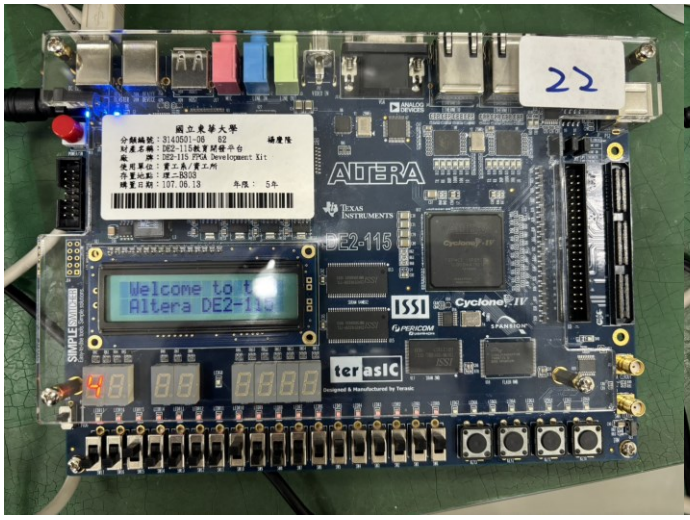
    mod9_adder m9(result, ledC, sw1, sw2, sw_cin);
    myDecoder dec(result, seg);
endmodule
```

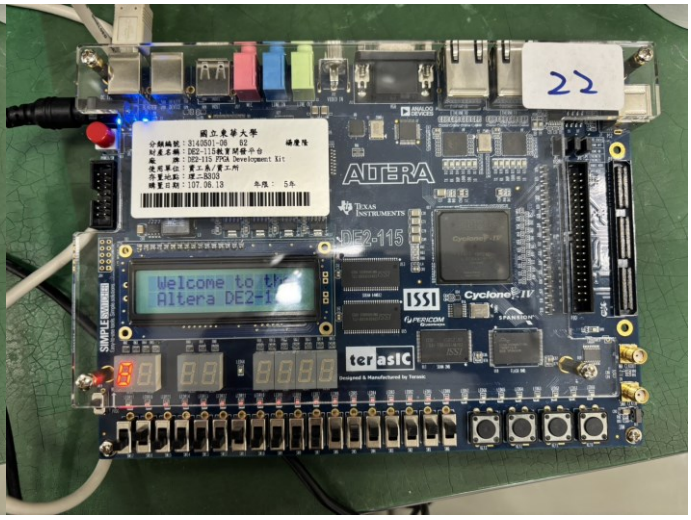
進位：顯示於 LED 燈
SUM：顯示於七段顯示器



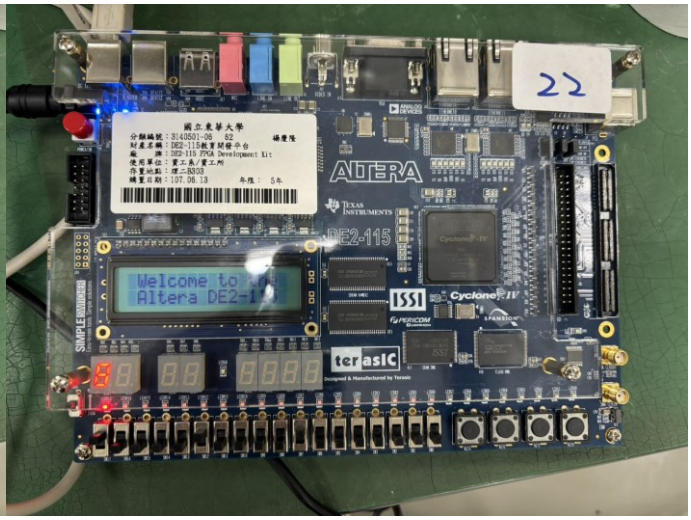
實作：初始狀態

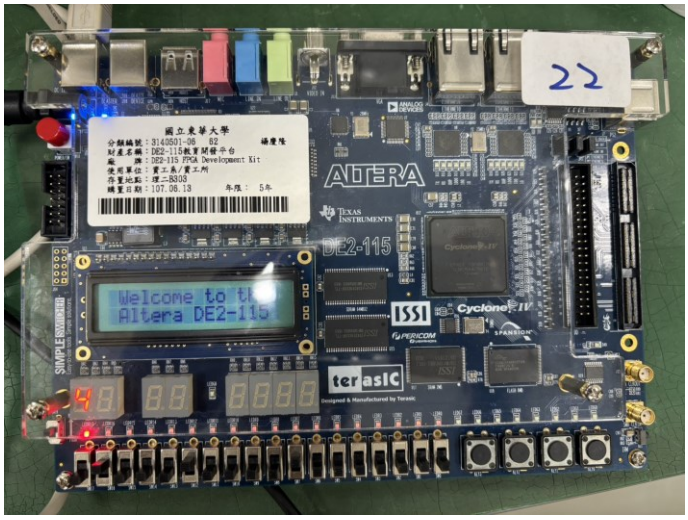

實作：1+1 = 2


實作：2+2 = 4


實作：4+4 = 8


實作：8+8 = 9(LED)+7 = 16


實作：8+8+1 = 9(LED)+8 = 17



實作：4+8+1 = 9(LED)+4 = 13